

35. MÉTODOS DE PRUEBA DEL SOFTWARE. FUNDAMENTOS. ESTRATEGIA DE PRUEBA DEL SOFTWARE: VERIFICACIÓN Y VALIDACIÓN. CAJA NEGRA Y CAJA BLANCA. PRUEBAS DE CONTORNO Y APLICACIONES ESPECIALIZADAS. PRUEBAS FUNCIONALES. PRUEBAS DE INTEGRACIÓN. PRUEBAS DE REGRESIÓN. PRUEBAS DE VALIDACIÓN.

Tema 35. Métodos de prueba del software. Fundamentos. Estrategia de prueba del software. Verificación y validación. Caja negra y caja blanca. Pruebas de entorno y aplicaciones especializadas. Pruebas funcionales. Pruebas de integración. Pruebas de regresión. Pruebas de validación.

35.1 Métodos de prueba del software. Fundamentos. Verificación y validación

35.1.1 Principios de las pruebas

35.2 Estrategias de prueba del software

35.2.1 Pruebas de Unidad

35.2.2 Pruebas de integración.

35.2.3 Pruebas de regresión.

35.2.4 Pruebas del sistema

35.2.5 Pruebas de implantación

35.2.6 Pruebas de Validación.

35.3 Pruebas de Caja negra y Caja blanca

35.3.1 Pruebas de Caja blanca

35.3.1.1 Prueba del camino básico

35.3.1.2 Prueba de la estructura de control

Prueba de condiciones

Prueba de flujo de datos

Prueba de bucles

35.3.2 Pruebas de Caja Negra

35.3.2.1 Partición equivalente

35.3.2.2 Análisis de valores límite

35.3.2.3 Valores típicos de error y valores imposibles

35.3.2.4 Basados en grafos

35.3.2.5 Tabla Ortogonal

35.3.2.6 Prueba de comparación

35.4 Pruebas de entorno y aplicaciones especializadas

35.4.1. Prueba de interfaces gráficas de usuario

35.4.2 Prueba de arquitecturas cliente/servidor

35.4.3 Prueba de la documentación y facilidades de ayuda

35.4.4 Prueba de sistemas de tiempo-real

35.5 Pruebas funcionales

35.1 Fundamentos de las pruebas del software. Verificación y Validación

La prueba del software es un elemento de un tema más amplio que, a menudo, es conocido como **verificación** y **validación** (V&V). La **verificación** se refiere al conjunto de actividades que aseguran que el software implementa correctamente una función específica. La **validación** se refiere a un conjunto diferente de actividades que aseguran que el software construido se ajusta a los requisitos del cliente.

- **Verificación:** ¿Estamos construyendo el producto correctamente
- **Validación:** ¿Estamos construyendo el producto correcto?

La verificación y la validación abarcan una amplia lista de actividades de Garantía de Calidad del Software (SQA) que incluye: revisiones técnicas formales, auditorías de calidad y de configuración, monitorización de rendimientos, simulación, estudios de factibilidad, revisión de la documentación, revisión de la base de datos, análisis algorítmico, pruebas de desarrollo, pruebas de validación y pruebas de instalación. A pesar de que las actividades de prueba tienen un papel muy importante en la Verificación y Validación, muchas otras actividades son también necesarias. La aplicación adecuada de los métodos y de las herramientas, las revisiones técnicas formales efectivas y una sólida gestión y medición, conducen a la calidad, que se confirma durante las pruebas.

La prueba presenta una anomalía para el ingeniero del software, ya que, mientras en las fases anteriores de definición y desarrollo, el ingeniero intenta construir, al llegar las pruebas, el ingeniero diseña una serie de casos de prueba que pretenden “demoler” lo construido. Por eso los autores dicen que la prueba se puede ver como destructiva, en lugar de constructiva.

El proceso de pruebas del software tiene dos objetivos distintos:

- Para demostrar al desarrollador y al cliente que el software satisface sus requerimientos. Para el software a medida, esto significa que debería haber al menos una prueba para cada requerimiento de los documentos de requerimientos del sistema y del usuario. Para productos de software genéricos, significa que debería haber pruebas para todas las características del sistema que se incorporarán en la entrega del producto.
- Para descubrir errores en el software en que el comportamiento de éste es incorrecto, no deseable o no cumple su especificación. La prueba de errores está relacionada con la eliminación de todos los tipos de comportamientos del sistema no deseables, tales como caídas del sistema, interacciones no permitidas con otros sistemas, cálculos incorrectos y corrupción de datos.

De las muchas definiciones válidas de la “prueba del software” podemos quedarnos con:

- La prueba es el proceso de ejecución de un programa con la intención de descubrir errores (Glenford Myers).
- La prueba es cualquier actividad dirigida a evaluar un la capacidad de un programa y determinar que alcanza los resultados requeridos.

35.1.1 Principios de las pruebas

Conocida la definición de prueba, algunos de los principios que les afectan son:

- La prueba es el proceso de ejecutar un programa con la intención de descubrir errores, por lo que un buen caso de prueba es aquel que tiene una alta probabilidad de descubrir un error no encontrado hasta entonces.
- Las pruebas deben planificarse antes de que se empiece a codificar. Así la planificación comienza con el modelo de requisitos, y la definición detallada de los casos de prueba una vez que el diseño del sistema está consolidado.
- El 80% de los errores estarán localizados en el 20% de los módulos.
- La prueba completa es imposible.
- Para ser más eficaces, las pruebas deberían ser hechas por un equipo independiente.
- La probabilidad de la existencia de más errores en una parte del software es proporcional al número de errores ya encontrados en dicha parte.

35.2 Estrategias de prueba del software.

Una estrategia de prueba del software integra las técnicas de diseño de casos de prueba en una serie de pasos bien planificados que dan como resultado una correcta construcción del software. La estrategia proporciona un mapa que describe los pasos que hay que llevar a cabo como parte de la prueba, cuándo se deben planificar y realizar esos pasos, y cuánto esfuerzo, tiempo y recursos se van a requerir. Por tanto, cualquier estrategia de prueba debe incorporar la planificación de la prueba, el diseño de casos de prueba, la ejecución de las pruebas y la agrupación y evaluación de los datos resultantes.

Una estrategia de prueba del software debe ser suficientemente flexible para promover la creatividad y la adaptabilidad necesarias para adecuar la prueba a todos los grandes sistemas basados en software. Al mismo

tiempo, la estrategia debe ser suficientemente rígida para promover un seguimiento razonable de la planificación y la gestión a medida que progresa el proyecto.

Se han propuesto varias estrategias de prueba del software en distintos libros. Todas proporcionan al ingeniero del software una plantilla para la prueba y todas tienen las siguientes características generales:

- Las pruebas comienzan a nivel de módulo y trabajan «hacia fuera», hacia la integración de todo el sistema.
- Según el momento, son apropiadas diferentes técnicas de prueba.
- La prueba la lleva a cabo el responsable del desarrollo del software y (para grandes proyectos) un grupo independiente de pruebas.
- La prueba y la depuración son actividades diferentes, pero la depuración se debe incluir en cualquier estrategia de prueba.

Una estrategia de prueba del software debe incluir pruebas de bajo nivel que verifiquen que todos los pequeños segmentos de código fuente se han implementado correctamente, así como pruebas de alto nivel que validen las principales funciones del sistema frente a los requisitos del cliente. Una estrategia debe proporcionar una guía al profesional y proporcionar un conjunto de hitos para el jefe de proyecto.

Se deben abordar los siguientes puntos si se desea implementar con éxito una estrategia de prueba del software:

- Especificar los requisitos del producto de manera cuantificable mucho antes de que comiencen las pruebas.
- Establecer los objetivos de la prueba de manera explícita.
- Comprender qué usuarios van a manejar el software y desarrollar un perfil para cada categoría de usuario.
- Desarrollar un plan de prueba que haga hincapié en la «prueba de ciclo rápido»
- Construir un software «robusto» diseñado para probarse a sí mismo

- Usar revisiones técnicas formales efectivas como filtro antes de la prueba.
- Llevar a cabo revisiones técnicas formales para evaluar la estrategia de prueba y los propios casos de prueba.
- Desarrollar un enfoque de mejora continua al proceso de prueba

Si consideramos el proceso desde el punto de vista procedimental, la prueba, en el contexto de la ingeniería del software, realmente es una serie de cinco pasos que se llevan a cabo secuencialmente. Inicialmente, la prueba se centra en cada módulo individualmente, asegurando que funcionan adecuadamente como una unidad. De ahí el nombre de *prueba de unidad*. La prueba de unidad hace un uso intensivo de las técnicas de prueba de caja blanca (se verán con posterioridad en otro apartado), ejercitando caminos específicos de la estructura de control del módulo para asegurar un alcance completo **y** una detección máxima de errores. A continuación, se deben ensamblar o integrar los módulos para formar el paquete de software completo. La *prueba de integración* se dirige a todos los aspectos asociados con el doble problema de verificación **y** de construcción del programa. Durante la integración, las técnicas que más prevalecen son las de diseño de casos de prueba de caja negra (se verán con posterioridad en otro apartado), aunque se pueden llevar a cabo algunas pruebas de caja blanca con el fin de asegurar que se cubren los principales caminos de control. Después de que el software se ha integrado (construido), se dirigen un conjunto de *pruebas de alto nivel*. Se debe comprobar que el software, al combinarlo con otros elementos del sistema (por ejemplo, hardware, gente, bases de datos), cada elemento encaja de forma adecuada y que se alcanza la funcionalidad y el rendimiento exigido. Esta es la base de *la prueba de sistemas*. Posteriormente se asegurará mediante la *prueba de implantación* el funcionamiento correcto del sistema integrado de hardware y software en el entorno de operación, y permitir al usuario que, desde el punto de vista de operación, realice la aceptación del sistema una vez instalado en su entorno real y en base al cumplimiento de

los requisitos no funcionales especificados. La *prueba de aceptación o de validación* proporciona una seguridad final de que el software satisface todos los requisitos funcionales, de comportamiento y de rendimiento. Durante las últimas tres pruebas (*sistemas, implantación y aceptación*) se usan exclusivamente técnicas de prueba de caja negra.

35.2.1 Pruebas de Unidad

Las pruebas unitarias tienen como objetivo verificar la funcionalidad y estructura de cada componente individualmente una vez que ha sido codificado.

La **prueba de unidad** es un proceso para probar los subprogramas, las subrutinas, los procedimientos individuales o las clases en un programa. Es decir, es mejor probar primero los bloques desarrollados más pequeños del programa, que inicialmente probar el software en su totalidad. Las motivaciones para hacer esto son tres. Primera, las pruebas de unidad son una manera de manejar los elementos de prueba combinados, puesto que se centra la atención inicialmente en unidades más pequeñas del programa. En segundo lugar, la prueba de una unidad facilita la tarea de eliminar errores (el proceso de establecer claramente y de corregir un error descubierto), puesto que, cuando se encuentra un error, se sabe que existe en un módulo particular. Finalmente, las pruebas de unidad introducen paralelismo en el proceso de pruebas del software presentándose la oportunidad de probar los múltiples módulos simultáneamente.

Se necesita dos tipos de información al diseñar los casos de prueba para una prueba de unidad: la especificación para el módulo y el código fuente del módulo. La especificación define típicamente los parámetros de entrada y de salida del módulo y su función.

Las pruebas de unidad son en gran parte orientadas a caja blanca. Una razón es que como en pruebas de entidades más grandes tales como programas enteros (es el caso para los procesos de prueba subsecuentes), la prueba de caja blanca llega a ser menos factible. Una segunda razón es que los procesos de prueba subsecuentes están orientados a encontrar diversos tipos de errores. Por lo tanto, el procedimiento para el diseño de casos de prueba para una prueba de unidad es la siguiente: analizar la lógica del módulo usando uno o más de los métodos de caja blanca y después completar los casos de prueba aplicando métodos de caja negra a la especificación del módulo.

35.2.2 Pruebas de integración.

El objetivo de las **pruebas de integración** es verificar el correcto ensamblaje entre los distintos componentes una vez que han sido probados unitariamente con el fin de comprobar que interactúan correctamente a través de sus interfaces, tanto internas como externas, cubren la funcionalidad establecida y se ajustan a los requisitos no funcionales especificados en las verificaciones correspondientes.

En las pruebas de integración se examinan las interfaces entre grupos de componentes o subsistemas para asegurar que son llamados cuando es necesario y que los datos o mensajes que se transmiten son los requeridos. Debido a que en las pruebas unitarias es necesario crear módulos auxiliares que simulen las acciones de los componentes invocados por el que se está probando y a que se han de crear componentes "conductores" para establecer las precondiciones necesarias, llamar al componente objeto de la prueba y examinar los resultados de la prueba, a menudo se combinan los tipos de prueba unitarias y de integración.

Los tipos fundamentales de integración son los siguientes:

- *Integración incremental:* se combina el siguiente componente que se debe probar con el conjunto de componentes que ya están probados y se va incrementando progresivamente el número de componentes a probar. Con el tipo de prueba incremental lo más probable es que los problemas que surjan al incorporar un nuevo componente o un grupo de componentes previamente probado, sean debidos a este último o a las interfaces entre éste y los otros componentes.
- *Integración no incremental:* se prueba cada componente por separado y posteriormente se integran todos de una vez realizando las pruebas pertinentes. Este tipo de integración se denomina también **Big-Bang**.

Dentro de la *integración incremental*, tenemos tres tipos de estrategias:

- *Estrategia Descendente (top-down):* El primer componente que se prueba es el primero de la jerarquía. Los componentes de nivel más bajo se sustituyen por componentes auxiliares llamados **resguardos** para simular a los componentes invocados. Luego se van sustituyendo los resguardos subordinados por los componentes reales. Ventajas: Las interfaces entre los distintos componentes se prueban en una fase temprana y con frecuencia. Verifica los puntos de decisión o de control principales al principio del proceso de prueba.
- *Estrategia Ascendente (bottom-up):* En este caso se crean primero los componentes de más bajo nivel y se crean componentes **conductores o controladores** para simular a los componentes que los llaman. A continuación se sustituyen los controladores por los módulos desarrollados de más alto nivel y se prueban.
- *Estrategia combinada:* A menudo es útil aplicar las estrategias anteriores conjuntamente. De este modo, se prueban las partes principales del sistema con un enfoque **top-down**, mientras que las

partes de nivel más bajo se prueban siguiendo un enfoque **bottom-up**.

35.2.3 Pruebas de regresión

Cada vez que se añade un nuevo módulo como parte de una prueba de integración, el software cambia. Se establecen nuevos caminos de flujo de datos, pueden ocurrir nuevas E/S y se invoca una nueva lógica de control. Estos cambios pueden causar problemas con funciones que antes trabajaban perfectamente. En este contexto, la **prueba de regresión** consiste en volver a ejecutar un subconjunto de pruebas que se han llevado a cabo anteriormente para asegurarse de que los cambios no han propagado efectos colaterales no deseados.

En un contexto más amplio, las pruebas con éxito (de cualquier tipo) dan como resultado el descubrimiento de errores, y los errores hay que corregirlos. Cuando se corrige el software, se cambia algún aspecto de la configuración del software (el programa, su documentación o los datos que lo soportan). La prueba de regresión es la actividad que ayuda a asegurar que los cambios (debidos a las pruebas o por otros motivos) no introducen un comportamiento no deseado o errores adicionales. La prueba de regresión se puede hacer manualmente, volviendo a realizar un subconjunto de todos los casos de prueba o utilizando herramientas automáticas de reproducción de captura. Las herramientas de reproducción de captura permiten al ingeniero del software capturar casos de prueba y los resultados para la subsiguiente reproducción y comparación.

35.2.4 Pruebas de sistema

Las pruebas del sistema tienen como objetivo ejercitar profundamente el sistema comprobando la integración del sistema de información

globalmente, verificando el funcionamiento correcto de las interfaces entre los distintos subsistemas que lo componen y con el resto de sistemas de información con los que se comunica.

Una vez que se han probado los componentes individuales y se han integrado, se prueba el sistema de forma global. En esta etapa pueden distinguirse los siguientes tipos de pruebas, cada uno con un objetivo claramente diferenciado:

- **Pruebas funcionales.** Dirigidas a asegurar que el sistema de información realiza correctamente todas las funciones que se han detallado en las especificaciones dadas por el usuario del sistema.
- **Pruebas de comunicaciones.** Determinan que las interfaces entre los componentes del sistema funcionan adecuadamente, tanto a través de dispositivos remotos, como locales. Asimismo, se han de probar las interfaces hombre/máquina.
- **Pruebas de rendimiento.** Consisten en determinar que los tiempos de respuesta están dentro de los intervalos establecidos en las especificaciones del sistema.
- **Pruebas de volumen.** Consisten en examinar el funcionamiento del sistema cuando está trabajando con grandes volúmenes de datos, simulando las cargas de trabajo esperadas.
- **Pruebas de sobrecarga.** Consisten en comprobar el funcionamiento del sistema en el umbral límite de los recursos, sometiéndole a cargas masivas. El objetivo es establecer los puntos extremos en los cuales el sistema empieza a operar por debajo de los requisitos establecidos.
- **Pruebas de disponibilidad de datos.** Consisten en demostrar que el sistema puede recuperarse ante fallos, tanto de equipo físico como lógico, sin comprometer la integridad de los datos.

- **Pruebas de facilidad de uso.** Consisten en comprobar la adaptabilidad del sistema a las necesidades de los usuarios, tanto para asegurar que se acomoda a su modo habitual de trabajo, como para determinar las facilidades que aporta al introducir datos en el sistema y obtener los resultados.
- **Pruebas de operación.** Consisten en comprobar la correcta implementación de los procedimientos de operación, incluyendo la planificación y control de trabajos, arranque y re arranque del sistema, etc.
- **Pruebas de entorno.** Consisten en verificar las interacciones del sistema con otros sistemas dentro del mismo entorno.
- **Pruebas de seguridad.** Consisten en verificar los mecanismos de control de acceso al sistema para evitar alteraciones indebidas en los datos.
- **Pruebas de configuración.** Programas tales como sistemas operativos, sistemas de gerencia de base de datos, y programas de conmutación de mensajes soportan una variedad de configuraciones de hardware, incluyendo varios tipos y números de dispositivos de entrada-salida y líneas de comunicaciones, o diversos tamaños de memoria. A menudo el número de configuraciones posibles es demasiado grande para probar cada uno, pero en lo posible, se debe probar el programa con cada tipo de dispositivo de hardware y con la configuración mínima y máxima. Si el programa por sí mismo se puede configurar para omitir componentes, o si puede funcionar en diversas computadoras, cada configuración posible de este debe ser probada.
- **Pruebas de instalación.** Al funcionar incorrectamente el programa de instalación podría evitar que el usuario tenga una experiencia acertada con el sistema. La primera experiencia de un usuario es cuando él o ella instala la aplicación. Si esta fase se realiza mal, entonces el usuario/el

cliente puede buscar otro producto o tener poca confianza en la validez de la aplicación.

- **Pruebas de documentación.** La documentación del usuario debe ser el tema de una inspección, comprobándola para saber si hay exactitud y claridad. Cualquiera de los ejemplos ilustrados en la documentación se deben probar y hacer parte de los casos y alimentarlos al programa.

35.2.5 Prueba de implantación

El objetivo de las **pruebas de implantación** es comprobar el funcionamiento correcto del sistema integrado de hardware y software en el entorno de operación, y permitir al usuario que, desde el punto de vista de operación, realice la aceptación del sistema una vez instalado en su entorno real y en base al cumplimiento de los requisitos no funcionales especificados.

Una vez que hayan sido realizadas las pruebas del sistema en el entorno de desarrollo, se llevan a cabo las verificaciones necesarias para asegurar que el sistema funcionará correctamente en el entorno de operación. Debe comprobarse que responde satisfactoriamente a los requisitos de rendimiento, seguridad, operación y coexistencia con el resto de los sistemas de la instalación para conseguir la aceptación del usuario de operación.

Las pruebas de seguridad van dirigidas a verificar que los mecanismos de protección incorporados al sistema cumplen su objetivo; las de rendimiento a asegurar que el sistema responde satisfactoriamente en los márgenes establecidos en cuanto tiempos de respuesta, de ejecución y de utilización de recursos, así como los volúmenes de espacio en disco y capacidad; por último con las pruebas de operación se comprueba que la planificación y control de trabajos del sistema se realiza de acuerdo a los procedimientos

establecidos, considerando la gestión y control de las comunicaciones y asegurando la disponibilidad de los distintos recursos.

Asimismo, también son llevadas a cabo las pruebas de gestión de copias de seguridad y recuperación, con el objetivo de verificar que el sistema no ve comprometido su funcionamiento al existir un control y seguimiento de los procedimientos de salvaguarda y de recuperación de la información, en caso de caídas en los servicios o en algunos de sus componentes. Para comprobar estos últimos, se provoca el fallo del sistema, verificando si la recuperación se lleva a cabo de forma apropiada. En el caso de realizarse de forma automática, se evalúa la inicialización, los mecanismos de recuperación del estado del sistema, los datos y todos aquellos recursos que se vean implicados.

Las verificaciones de las pruebas de implantación y las pruebas del sistema tienen muchos puntos en común al compartir algunas de las fuentes para su diseño como pueden ser los casos para probar el rendimiento (pruebas de sobrecarga o de stress).

El responsable de implantación junto al equipo de desarrollo determina las verificaciones necesarias para realizar las pruebas así como los criterios de aceptación del sistema. Estas pruebas las realiza el equipo de operación, integrado por los técnicos de sistemas y de operación que han recibido previamente la formación necesaria para llevarlas a cabo.

35.2.6 Pruebas de Validación

El objetivo de las **pruebas de validación**, también conocidas como **pruebas de aceptación** es validar que un sistema cumple con el funcionamiento esperado y permitir al usuario de dicho sistema que determine su aceptación desde el punto de vista de su funcionalidad y rendimiento.

Las pruebas de validación son definidas por el usuario del sistema y preparadas por el equipo de desarrollo, aunque la ejecución y aprobación final corresponden al usuario. Estas pruebas van dirigidas a comprobar que el sistema cumple los requisitos de funcionamiento esperado, recogidos en el catálogo de requisitos y en los criterios de aceptación del sistema de información, y conseguir así la aceptación final del sistema por parte del usuario.

El responsable de usuarios debe revisar los criterios de aceptación que se especificaron previamente en el plan de pruebas del sistema y, posteriormente, dirigir las pruebas de aceptación final. La validación del sistema se consigue mediante la realización de pruebas de caja negra que demuestran la conformidad con los requisitos y que se recogen en el plan de pruebas, el cual define las verificaciones a realizar y los casos de prueba asociados. Dicho plan está diseñado para asegurar que se satisfacen todos los requisitos funcionales especificados por el usuario teniendo en cuenta también los requisitos no funcionales relacionados con el rendimiento, seguridad de acceso al sistema, a los datos y procesos, así como a los distintos recursos del sistema.

La formalidad de estas pruebas dependerá en mayor o menor medida de cada organización, y vendrá dada por la criticidad del sistema, el número de usuarios implicados en las mismas y el tiempo del que se disponga para llevarlas cabo, entre otros.

Si el software se desarrolla como un producto que va a ser usado por muchos clientes, no es práctico realizar pruebas de aceptación formales para cada uno de ellos. La mayoría de los desarrolladores de productos de software llevan a cabo un proceso denominado prueba alfa y beta para descubrir errores que parezca que sólo el usuario final puede descubrir.

La **prueba alfa** se lleva a cabo, por un cliente, en el lugar de desarrollo. Se usa el software de forma natural con el desarrollador como observador del

usuario y registrando los errores y los problemas de uso. Las pruebas alfa se llevan a cabo en un entorno controlado.

La **prueba beta** se lleva a cabo por los usuarios finales del software en los lugares de trabajo de los clientes. A diferencia de la prueba alfa, el desarrollador no está presente normalmente. Así, la prueba beta es una aplicación «en vivo» del software en un entorno que no puede ser controlado por el desarrollador. El cliente registra todos los problemas (reales o imaginarios) que encuentra durante la prueba beta e informa a intervalos regulares al desarrollador. Como resultado de los problemas informados durante la prueba beta, el desarrollador del software lleva a cabo modificaciones y así prepara una versión del producto de software para toda la clase de clientes.

35.3 Pruebas de Caja Negra y Caja Blanca.

Cualquier producto de ingeniería se puede probar de una de estas dos formas:

- Conociendo la función específica para la que fue diseñado el producto, se pueden llevar a cabo pruebas que demuestren que cada función es completamente operativa.
- Conociendo el funcionamiento del producto, se pueden desarrollar pruebas que aseguren que «todas las piezas encajan», o sea, que la operación interna se ajusta a las especificaciones y que todos los componentes internos se han comprobado de forma adecuada.

El primer enfoque de prueba se denomina prueba de **caja negra** y el segundo, prueba de **caja blanca**.

La prueba de **caja negra** se refiere a las pruebas que se llevan a cabo sobre la interfaz del software. O sea, los casos de prueba pretenden demostrar que las funciones del software son operativas, que la entrada se

acepta de forma adecuada y que se produce un resultado correcto, así como que la integridad de la información externa (por ejemplo, archivos de datos) se mantiene. Una prueba de caja negra examina algunos aspectos del modelo fundamental del sistema sin tener mucho en cuenta la estructura lógica interna del software.

La prueba de **caja blanca** del software se basa en el minucioso examen de los detalles procedimentales. Se comprueban los caminos lógicos del software proponiendo casos de prueba que ejerciten conjuntos específicos de condiciones y/o bucles. Se puede examinar el «estado del programa» en varios puntos para determinar si el estado real coincide con el esperado o mencionado.

35.3.1 Pruebas de caja blanca

El enfoque de la estrategia de pruebas conocido por el nombre de método de «**caja blanca**» o, también, «**conducido por la lógica**» o «**logic driven**», se centra en probar el comportamiento interno y la estructura del programa, examinando la lógica interna del mismo y sin considerar los aspectos de rendimiento. El objetivo de este enfoque es ejecutar, al menos una vez, todas las sentencias y ejecutar todas las condiciones tanto en su vertiente verdadera como falsa, teniendo en cuenta que la única información de entrada con que se cuenta es el diseño del programa y el código fuente. Las técnicas específicas más usuales que siguen el método de «caja blanca» son:

- Prueba del camino básico
- Prueba de la estructura de control
 - o Prueba de condiciones
 - o Prueba de flujo de datos
 - o Prueba de bucles

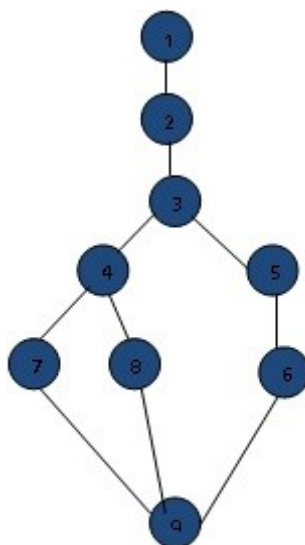
35.3.1.1. Prueba del camino básico

Fue propuesta inicialmente por Tom McCabe. Permite al diseñador de casos de prueba obtener una medida de la complejidad lógica de un diseño procedimental y usar esa medida como guía para la definición de un conjunto básico de caminos de ejecución. Los casos de prueba obtenidos del conjunto básico garantizan que durante la prueba se ejecuta por lo menos una vez cada sentencia del programa.

De la técnica del camino básico se derivan dos pruebas complementarias y no excluyentes:

- *Prueba de cobertura de sentencias.* Consiste en generar casos de prueba que permitan probar todas y cada una de las sentencias de un módulo una vez. Esta prueba es necesaria pero no es suficiente.
- *Prueba de cobertura de condiciones.* Consiste en diseñar juegos de prueba que consideren todos los valores posibles de cada una de las condiciones. Esta prueba también es necesaria pero no es suficiente ya que no garantiza que todos los caminos sean cubiertos, por lo que debe ser complementada con la anterior.

La técnica de prueba del camino básico utiliza una convención de representación denominada grafos de flujo para la determinación de caminos y para ello se apoya en el concepto de complejidad ciclomática propuesto por Tom McCabe.



La notación de **grafo de flujo**, propuesta por McCabe se utiliza para simplificar el desarrollo del conjunto básico de caminos de ejecución. Su objetivo es ejemplificar el flujo de control del módulo que se está probando y para ello utiliza tres elementos:

- **Nodos** o tareas de procesamiento (N). Representan cero, una o varias sentencias procedimentales. Cada nodo comprende como máximo una sentencia de decisión (bifurcación). Cada nodo que contiene una condición se denomina **nodo predicado** y está caracterizado porque dos o más aristas emergen de el.
- **Aristas**, flujo de control o conexiones (A). Unen dos nodos, incluso aunque el nodo no represente ninguna sentencia procedimental.
- **Regiones** (R). Son las áreas delimitadas por las aristas y nodos. Cuando se contabilizan las regiones debe incluirse el área externa como una región más.

La **complejidad ciclomática** es una métrica del software basada en la teoría de grafos, que proporciona una medición cuantitativa de la complejidad lógica de un programa. Cuando se usa en el contexto del método de prueba del camino básico, el valor calculado como complejidad ciclomática define el número de caminos independientes del conjunto

básico de un programa y nos da un límite superior para el número de pruebas que se deben realizar para asegurar que se ejecuta cada sentencia al menos una vez. Un camino independiente es cualquier camino del programa que introduce, por lo menos, un nuevo conjunto de sentencias de proceso o una nueva condición. En términos del grafo de flujo, un camino independiente está constituido por lo menos por una arista que no haya sido recorrida anteriormente a la definición del camino. El valor de la complejidad se puede obtener de tres formas:

- 1- El número de regiones del grafo. (R)
- 2- El número de aristas menos el número de nodos + 2. ($A - N + 2$.)
- 3- Número de nodos predicado + 1. ($P + 1$)

La prueba del camino básico nos permite obtener los casos de prueba de la siguiente forma:

- 1- Usando el diseño o el código como base, dibujamos el correspondiente grafo de flujo.
- 2- Determinamos la complejidad ciclomática del grafo de flujo resultante.
- 3- Determinamos un conjunto básico de caminos linealmente independientes.
- 4- Preparamos los casos de prueba que forzarán la ejecución de cada camino del conjunto básico.

35.3.1.2 Pruebas de la estructura de control

Dentro de éste tipo de prueba se contempla el método del camino básico mencionado anteriormente pero además existen otras pruebas asociadas que permiten ampliar la cobertura de la prueba y mejorar su calidad.

35.3.1.2.1 Prueba de condición.

Es un método de diseño de casos de prueba que ejercita las condiciones lógicas contenidas en el módulo de un programa. Algunos conceptos empleados alrededor de esta prueba son los siguientes:

- *Condición simple*: es una variable lógica o una expresión relacional ($E_1 < \text{operador} - \text{relacional} > E_2$).
- *Condición compuesta*: esta formada por dos o más condiciones simples, operadores lógicos y paréntesis.

En general los tipos de errores que se buscan en una prueba de condición, son los siguientes:

- *Error en operador lógico* (existencia de operadores lógicos incorrectos, desaparecidos, sobrantes).
- *Error en variable lógica*.
- *Error en paréntesis lógico*.
- *Error en operador relacional*.
- *Error en expresión aritmética*.

35.3.1.2.2 Prueba del flujo de datos.

Selecciona caminos de prueba de un programa de acuerdo con la ubicación de las definiciones y los usos de las variables del programa.

35.3.1.2.3. Prueba de bucles

La prueba de bucles es una técnica de prueba de caja blanca que se centra exclusivamente en la validez de las construcciones de bucles. Se pueden definir cuatro clases diferentes de bucles:

- **Bucles simples.** A los bucles simples se les debe aplicar el siguiente conjunto de pruebas, donde **n** es el número máximo de pasos permitidos por el bucle:
 1. pasar por alto totalmente el bucle
 2. pasar una sola vez por el bucle
 3. pasar dos veces por el bucle
 4. hacer m pasos por el bucle con $m < n$
 5. hacer $n - 1$, n y $n + 1$ pasos por el bucle
- **Bucles anidados.** Si extendiéramos el enfoque de prueba de los bucles simples a los bucles anidados, el número de posibles pruebas aumentaría geométricamente a medida que aumenta el nivel de anidamiento. Esto llevaría un número impracticable de pruebas. Se sugiere un enfoque que ayuda a reducir el número de pruebas:
 1. Comenzar por el bucle más interior. Establecer o configurar los demás bucles con sus valores mínimos.
 2. Llevar a cabo las pruebas de bucles simples para el bucle más interior, mientras se mantienen los parámetros de iteración (por ejemplo, contador del bucle) de los bucles externos en sus valores mínimos. Añadir otras pruebas para valores fuera de rango o excluidos.
 3. Progresar hacia fuera, llevando a cabo pruebas para el siguiente bucle, pero manteniendo todos los bucles externos en sus valores mínimos y los demás bucles anidados en sus valores «típicos».
 4. Continuar hasta que se hayan probado todos los bucles.
- **Bucles concatenados.** Los bucles concatenados se pueden probar mediante el enfoque anteriormente definido para los bucles simples, mientras cada uno de los bucles sea independiente del resto. Sin embargo, si hay dos bucles concatenados y se usa el controlador del bucle 1 como valor inicial del bucle 2, entonces los bucles no son

independientes. Cuando los bucles no son independientes, se recomienda usar el enfoque aplicado para los bucles anidados.

- **Bucles no estructurados.** En este caso, ante la complejidad que puede representar la comprensión del flujo de control, es más práctico rediseñar el módulo a probar de forma que se codifique mediante bucles estructurados.

35.3.2 Pruebas de caja negra.

El enfoque de la estrategia de pruebas conocido por el nombre de método de «**caja negra**» o, también, «**conducido por los datos**» («**data driven**») o «**conducido por la entrada/salida**» («**input-output driven**»), o también **pruebas de comportamiento**, no considera el detalle procedimental de los programas y se centra en buscar situaciones donde el programa no se ajusta a su especificación, utilizando ésta como entrada para derivar los casos de prueba.

Si por las especificaciones funcionales se sabe lo que tiene que hacer un módulo, es más sencillo comprobarlo que desmenuzarlo y examinarlo internamente en todas las circunstancias posibles. Por ello, las pruebas de «caja negra» son el enfoque más simple de prueba del software, y en ella diseñaremos los casos de prueba a partir de las especificaciones funcionales.

En este tipo de pruebas los casos de prueba consisten en conjuntos de datos de entrada que deberán generar una salida acorde con la especificación. La atención se centra, pues, en los datos de entrada y salida ignorando intencionadamente el conocimiento del código del programa. Si con esta técnica se quisieran encontrar todos los errores del programa, habría que recurrir a probar todas las posibles combinaciones de casos de entrada, lo que supondría generar todas las posibles

combinaciones de valores para todas las posibles variables de entrada, y ello, en la realidad, es imposible. De esta imposibilidad se pueden extraer la conclusión de que mediante el método de la «caja negra» no es posible asegurar que un programa esté libre de errores.

Dado que no se pueden probar todos los casos posibles, el método de «caja negra» contempla una serie de técnicas encaminadas a simplificar los casos de prueba. Éstas son:

- Partición equivalente
- El análisis de valores límite.
- Los valores típicos de error y los valores imposibles.
- Métodos de prueba basados en grafos.
- Prueba de la tabla ortogonal
- Las pruebas de comparación.

35.3.2.1 Partición equivalente.

La **partición equivalente** es un método de prueba de caja negra que divide el campo de entrada de un programa en clases de datos de los que se pueden derivar casos de prueba. Un caso de prueba ideal descubre de forma inmediata una clase de errores (por ejemplo, proceso incorrecto de todos los datos de carácter) que, de otro modo, requerirían la ejecución de muchos casos antes de detectar el error genérico. La partición equivalente se dirige a la definición de casos de prueba que descubran clases de errores, reduciendo así el número total de casos de prueba que hay que desarrollar.

El diseño de casos de prueba para la partición equivalente se basa en una evaluación de las clases de equivalencia para una condición de entrada. Una **clase de equivalencia** representa un conjunto de estados válidos o no válidos para condiciones de entrada. Típicamente, una condición de

entrada es un valor numérico específico, un rango de valores, un conjunto de valores relacionados o una condición lógica. Las clases de equivalencia se pueden definir de acuerdo con las siguientes directrices:

- 1- Si una condición de entrada especifica un rango, se define una clase de equivalencia válida y dos no válidas.
- 2- Si una condición de entrada requiere un valor específico, se define una clase de equivalencia válida y dos no válidas.
- 3- Si una condición de entrada especifica un miembro de un conjunto, se define una clase de equivalencia válida y una no válida.
- 4- Si una condición de entrada es lógica, se define una clase de equivalencia válida y una no válida.

35.3.2.2 Análisis de valores límite.

El **análisis de valores límite** es una técnica de diseño de casos de prueba que complementa a la partición de equivalencia y se justifica en la constatación de que para una condición de entrada que admite un rango de valores, es más fácil que existan errores en los límites que en el centro. Por tanto, la diferencia entre esta técnica y la partición de equivalencia estriba en que en el análisis de valores límite no se selecciona un elemento representativo de la clase de equivalencia, sino que se seleccionan uno o más elementos de manera que los límites de cada clase de equivalencia son objeto de prueba. Otra diferencia es que con esta técnica también se derivan casos de prueba para las condiciones de salida.

Al igual que en el caso anterior, la técnica de análisis de valores límite no asegura la prueba completa, ya que es imposible probar exhaustivamente todos los conjuntos de datos de entrada tanto en su vertiente válida como inválida. Sin embargo, la ventaja que presenta esta técnica es que maximiza el número de errores encontrados con el menor número de casos de prueba posibles, lo que rentabiliza la inversión efectuada en la prueba.

Las directrices de AVL son similares en muchos aspectos a las que proporciona la partición equivalente:

- 1- Si una condición de entrada especifica un rango delimitado por los valores a y b, se deben diseñar casos de prueba para los valores a y b, y para los valores justo por debajo y justo por encima de a y b, respectivamente.
- 2- Si una condición de entrada especifica un número de valores, se deben desarrollar casos de prueba que ejerciten los valores máximo y mínimo. También se deben probar los valores justo por encima y justo por debajo del máximo y del mínimo.
- 3- Aplicar las directrices 1 y 2 a las condiciones de salida.
- 4- Si las estructuras de datos internas tienen límites preestablecidos (por ejemplo, una matriz que tenga un límite definido de 100 entradas), hay que asegurarse de diseñar un caso de prueba que ejercite la estructura de datos en sus límites.

35.3.2.3. Valores típicos de error y valores imposibles.

Un buen complemento de las dos técnicas fundamentales de pruebas tipo «caja negra» (particiones de equivalencia y análisis de valores límite) consiste en incluir en los casos de prueba ciertos valores de los datos de entrada susceptibles de causar problemas, esto es, valores típicos de error, y valores especificados como no posibles, es decir, valores imposibles.

La determinación de los valores típicos de error se realiza en función de la naturaleza y funcionalidad del programa a probar, por lo que depende en buena medida de la experiencia del diseñador de la prueba.

Asimismo, dentro de las especificaciones del sistema o del programa a probar puede haber valores de datos especificados como no posibles. El hecho de probar estos valores imposibles se debe a que dichos valores podrían haber sido generados internamente por el sistema o el programa

provocando un mal funcionamiento del mismo. La prueba de valores imposibles debe realizarse siempre que dichos valores puedan ser detectados y el programa pueda manejarlos adecuadamente sin provocar errores irreparables.

35.3.2.4 Métodos de prueba basados en grafos.

En este método se debe entender los objetos (objetos de datos, objetos de programa tales como módulos o colecciones de sentencias del lenguaje de programación) que se modelan en el software y las relaciones que conectan a estos objetos. Una vez que se ha llevado a cabo esto, el siguiente paso es definir una serie de pruebas que verifiquen que todos los objetos tienen entre ellos las relaciones esperadas. En este método:

1. Se crea un grafo de objetos importantes y sus relaciones.
2. Se diseña una serie de pruebas que cubran el grafo de manera que se ejerciten todos los objetos y sus relaciones para descubrir errores.

Boris Beizer describe un número de modelados para pruebas de comportamiento que pueden hacer uso de los grafos:

- *Modelado del flujo de transacción.* Los nodos representan los pasos de alguna transacción (por ejemplo, los pasos necesarios para una reserva en una línea aérea usando un servicio en línea), y los enlaces representan las conexiones lógicas entre los pasos (por ejemplo, vuelo.información.entrada es seguida de validación /disponibilidad.procesamiento).
- *Modelado de estado finito.* Los nodos representan diferentes estados del software observables por el usuario (por ejemplo, cada una de las pantallas que aparecen cuando un telefonista coge una petición por teléfono), y los enlaces representan las transiciones que ocurren para moverse de estado a estado (por ejemplo, petición-información se

verifica durante inventario-disponibilidad-búsqueda y es seguido por cliente-factura-información-entrada).

- *Modelado de flujo de datos.* Los nodos objetos de datos y los enlaces son las transformaciones que ocurren para convertir un objeto de datos en otro.
- *Modelado de planificación.* Los nodos son objetos de programa y los enlaces son las conexiones secuenciales entre esos objetos. Los pesos de enlace se usan para especificar los tiempos de ejecución requeridos al ejecutarse el programa.
- *Gráfica Causa-efecto.* La gráfica Causa-Efecto representa una ayuda gráfica al seleccionar, de una manera sistemática, un gran conjunto de casos de prueba. Tiene un efecto secundario beneficioso en precisar estados incompletos y ambigüedades en la especificación. Un gráfico de causa-efecto es un lenguaje formal al cual se traduce una especificación. El gráfico es realmente un circuito de lógica digital (una red combinatoria de lógica), pero en vez de la notación estándar de la electrónica, se utiliza una notación algo más simple. No hay necesidad de tener conocimiento de electrónica con excepción de una comprensión de la lógica booleana (entendiendo los operadores de la lógica y, o, y no).

35.3.2.5 Prueba de la tabla ortogonal

Hay aplicaciones donde el número de parámetros de entrada es pequeño y los valores de cada uno de los parámetros están claramente delimitados. Cuando estos números son muy pequeños (por ejemplo, 3 parámetros de entrada tomando 3 valores diferentes), es posible considerar cada permutación de entrada y comprobar exhaustivamente el proceso del dominio de entrada. En cualquier caso, cuando el número de valores de entrada crece y el número de valores diferentes para cada elemento de dato se incrementa, la prueba exhaustiva se hace impracticable.

La prueba de la **tabla ortogonal** puede aplicarse a problemas en que el dominio de entrada es relativamente pequeño pero demasiado grande para posibilitar pruebas exhaustivas. El método de prueba de la tabla ortogonal es particularmente útil al encontrar errores asociados con fallos localizados -una categoría de error asociada con defectos de la lógica dentro de un componente software-. La prueba de tabla ortogonal permite proporcionar una buena cobertura de pruebas con bastantes menos casos de prueba que en la estrategia exhaustiva.

35.3.2.6 Prueba de comparación.

Hay situaciones en las que la fiabilidad del software es algo absolutamente crítico. En ese tipo de aplicaciones, a menudo se utiliza hardware y software redundante para minimizar la posibilidad de error. Cuando se desarrolla software redundante, varios equipos de ingeniería del software separados desarrollan versiones independientes de una aplicación, usando las mismas especificaciones. En esas situaciones, se deben probar todas las versiones con los mismos datos de prueba, para asegurar que todas proporcionan una salida idéntica. Luego, se ejecutan todas las versiones en paralelo y se hace una comparación en tiempo real de los resultados, para garantizar la consistencia.

Esas versiones independientes son la base de una técnica de prueba de caja negra denominada **prueba de comparación o prueba mano a mano**.

35.4 Pruebas de entorno y aplicaciones especializadas

A medida que el software de computadora se ha hecho más complejo, ha crecido también la necesidad de enfoques de pruebas especializados. Los métodos de prueba de caja blanca y de caja negra son aplicables a todos

los entornos, arquitecturas y aplicaciones pero a veces se necesitan unas directrices y enfoques más específicos para las pruebas.

35.4.1. Prueba de interfaces gráficas de usuario (IGUs)

Con el paso del tiempo la complejidad de las IGUs ha aumentado, originando más dificultad en el diseño y ejecución de los casos de prueba. Dado que las IGUs modernas tienen la misma apariencia y filosofía, se pueden obtener una serie de pruebas estándar. Los grafos de modelado de estado finito pueden ser utilizados para realizar pruebas que vayan dirigidas sobre datos específicos y programas objeto que sean relevantes para las IGUs. Considerando el gran número de permutaciones asociadas con las operaciones IGU, sería necesario utilizar herramientas automáticas para realizar las pruebas.

35.4.2 Prueba de arquitecturas cliente/servidor

La naturaleza distribuida de los entornos cliente/servidor, los aspectos de rendimiento asociados con el proceso de transacciones, la presencia potencial de diferentes plataformas hardware, las complejidades de las comunicaciones de red, la necesidad de servir a múltiples clientes desde una base de datos centralizada (o en algunos casos, distribuida) y los requisitos de coordinación impuestos al servidor se combinan todos para hacer las pruebas de la arquitectura **C/S** y el software residente en ellas, considerablemente más difíciles que la prueba de aplicaciones individuales.

35.4.3 Prueba de la documentación y facilidades de ayuda

Los errores en la documentación pueden ser tan destructivos para la aceptación del programa, como los errores en los datos o en el código fuente. Nada es más frustrante que seguir fielmente el manual de usuario y obtener resultados o comportamientos que no coinciden con los anticipados por el documento. La prueba de la documentación se puede enfocar en dos fases. La primera fase, la revisión e inspección, examina el

documento para comprobar la claridad editorial. La segunda fase, la prueba en vivo, utiliza la documentación junto al uso del programa real.

35.4.4 Prueba de sistemas de tiempo-real

La naturaleza asíncrona **y** dependiente del tiempo de muchas aplicaciones de tiempo real, añade un nuevo **y** potencialmente difícil elemento a la complejidad de las pruebas (el tiempo). El responsable del diseño de casos de prueba no sólo tiene que considerar los casos de prueba de caja blanca **y** de caja negra, sino también el tratamiento de sucesos (por ejemplo, procesamiento de interrupciones), la temporización de los datos **y** el paralelismo de las tareas (procesos) que manejan los datos. En muchas situaciones, los datos de prueba proporcionados al sistema de tiempo real cuando se encuentra en un determinado estado darán un proceso correcto, mientras que al proporcionárselos en otro estado llevarán a un error. Además, la estrecha relación que existe entre el software de tiempo real **y** su entorno de hardware también puede introducir problemas en la prueba. Se puede proponer una estrategia en cuatro pasos como método de prueba para sistemas de tiempo real:

- **Prueba de tareas.** El primer paso de la prueba de sistemas de tiempo real consiste en probar cada tarea independientemente.
- **Prueba de comportamiento.** Utilizando modelos del sistema creados con herramientas CASE, es posible simular el comportamiento del sistema en tiempo real y examinar su comportamiento como consecuencia de sucesos externos. Estas actividades de análisis pueden servir como base del diseño de casos de prueba que se llevan a cabo cuando se ha construido el software de tiempo real.
- **Prueba intertareas.** Una vez que se han aislado los errores en las tareas individuales y en el comportamiento del sistema, la prueba se dirige hacia los errores relativos al tiempo. Se prueban las tareas asíncronas que se sabe que comunican con otras, con diferentes

tasas de datos y cargas de proceso para determinar si se producen errores de sincronismo entre las tareas. Además, se prueban las tareas que se comunican mediante colas de mensajes o almacenes de datos, para detectar errores en el tamaño de esas áreas de almacenamiento de datos.

- **Prueba del sistema.** El software y el hardware están integrados, por lo que se lleva a cabo una serie de pruebas completas del sistema para intentar descubrir errores en la interfaz software/hardware.

35.5 Pruebas funcionales

En las pruebas funcionales se hace una verificación dinámica del comportamiento de un sistema, basada en la observación de un conjunto seleccionado de ejecuciones controladas o casos de prueba. Las pruebas funcionales son aquellas que se aplican al producto final, y permiten detectar en qué puntos el producto no cumple sus especificaciones, es decir, comprobar su funcionalidad. Para realizarlas se debe hacer una planificación que consiste en definir los aspectos a examinar y la forma de verificar su correcto funcionamiento, punto en el cual adquieren sentido los casos de prueba. Se usan sobre todo en el campo de la orientación a objetos.

Las formas actuales usadas para derivar casos de prueba funcionales son:

- La metodología SCENT permite derivar casos de prueba tomando como partida la definición de escenarios y actores que interactúan con el sistema, para luego definir prioridades, pasando por la elaboración de diagramas de dependencia, diagramas de estados y por último generar casos de prueba.
- Heumann desarrolla un método para generar casos de prueba tomando como base casos de uso, e identificando dentro de cada uno

los posibles escenarios, o caminos de ejecución, y por último definir los valores a probar de cada caso de prueba. Finalmente se obtiene una lista de casos de prueba, con los valores que deben probar y los resultados esperados para cada caso.

- La propuesta de Riebisch está centrada en la transformación automática de un modelo de casos de uso a un modelo de uso que sirve como entrada para realizar pruebas estadísticas automáticas, que mejoran el nivel de cobertura, partiendo de que diferentes partes de un software no necesitan ser probadas con la misma minuciosidad. El método comienza con el refinamiento de los casos de uso ampliándolos con precondiciones y postcondiciones, alternativas al camino de ejecución principal y referencia a otros casos de uso relacionados. Después se traducen a diagramas de estado y se elabora el modelo de uso donde se indica la probabilidad de que ocurra una transición y se identifican los caminos de ejecución más frecuentes. Por último, se extraen los modelos de prueba a partir de los modelos de uso y se generan recorridos aleatorios sobre cada modelo de uso. Cada camino aleatorio será un caso de prueba.
- Por último, Hartman propone una metodología centrada en dos productos: el primero compuesto por un modelo del sistema escrito en el lenguaje de modelado IF y un conjunto de diagramas UML de clases y estados que van a permitir la generación automática del conjunto de pruebas. El segundo, conformado por un conjunto de objetos de casos de prueba ejecutables tanto en el modelo del sistema como en la implantación, lo que permite comparar los resultados esperados y los obtenidos. Para obtener los productos enunciados, en primer lugar se construye un modelo de comportamiento del sistema a partir de sus especificaciones. Este modelo está compuesto por diagramas UML de clases y un diagrama UML de estados por cada clase que describe el comportamiento de los objetos de dicha clase. A continuación se elaboran los objetivos de

las pruebas (pruebas de casos de uso con datos concretos, pruebas de carga del sistema, etc.) y se traducen a un conjunto de directivas de generación y ejecución de pruebas. En el siguiente paso, una herramienta genera automáticamente una serie de pruebas que satisfacen los objetivos de prueba anteriores y se ejecuta automáticamente. Por último, se analizan los resultados y se repiten los pasos hasta que se alcanzan los objetivos deseados.

Bibliografía

- Ingeniería del Software. Un enfoque práctico. Roger Pressman. Ed. Mc Graw Hill.
- Ingeniería del Software. Ian Sommerville. Ed. PEARSON ADDISON WESLEY.
- Metodología MÉTRICA versión 3. Técnicas y Prácticas. Ministerio de Administraciones Públicas.
- Grupo ARQUISOFT - Johanna Rojas - Emilio Barrios – 2007
- Método para generar casos de prueba funcional en el desarrollo de software. Liliana Gonzalez Palacio.

Autor: Hernán Vila Pérez

Jefe del Servicio de Informática. Instituto Galego de Vivenda e Solo
Vicepresidente del CPEIG

SEGURIDAD EN LOS SISTEMAS DE INFORMACIÓN